

Sorbonne Université

Implementation of finite element methods (MU5MAM30)

Some project proposals

[**Note** : links doesn't seem highlighted nor functional from the Github PDF reader, download and use a proper PDF reader]

Instructions

- Projects can be tackled in groups of at most three students (but different groups can work independently on the same subject).
- Reports should be brief and to the point, and ideally typeset in Latex or Markdown. They must be accompanied with source code and build instructions, and either uploaded on the git repo in your branch (say inside the "projects" directory) or sent to me by email, in the latter case in a single archive (zip or equivalent) with base name format `name_surname-MU5MAM30-project`.
- When your project is finished or you have scheduled a deadline for it, take an appointment (30min slot, on an individual basis even if working in group) by sending me an e-mail listing your availabilities in the foreseen window for the presentation (select one to two weeks), I will then answer with a slot proposal.
- It is not needed to prepare slides for the presentation, I will have read your code and report in advance. You will expose me the difficulties and achievements during 5 to 10 minutes, and then I will ask you some specific questions related to your submission, or slightly around it, to evaluate your scientific autonomy.

Since most of the projects are extensions in different directions of materials that we treated during the class, you *might* combine more than one to build up your (multi)project.

In case of guidance needed, for the choice or the realization of the project, contact me by e-mail or show-up in my office in 16-26 325. Proposing an alternate project is also accepted, but you must send me the proposal for agreement first.

Project 1 : $\mathbb{P}_2$ -Lagrange finite elements

For this project you are asked to implement a $\mathbb{P}_2$ -Lagrange FEM version of what we have been doing during the tutorials with $\mathbb{P}_1$ elements only.

After computing the mass and stiffness terms for the reference element, which shouldn't be too complicated, the main challenge is to devise well the “local to global” numbering of the degrees of freedom, which are no longer simply the mesh vertices.

In terms of git repository, you could :

- create files `include/fem/P2.h` and `/src/fem/P2.cpp` along the existing $\mathbb{P}_1$ versions;
- modify `include/fem/mass.h` and `include/fem/stiffness.h` to take into account both FEM versions. Alternatively, replace them with 4 files `mass_Pi.h` and `stiffness_Pi.h` with $i = 1$ and $i = 2$;
- modify `include/FEM/poisson.h` and `include/FEM/navier_stokes.h` (and of course also the `src` versions) to add an argument describing the FEM type used for the solver. I'd suggest to do it in the constructor of the classes `PoissonSolver` and `NavierStokesSolver`, in addition to the already existing mesh argument;
- finally add this FEM type choice in the test executables located in `src/bin`, for example as command line arguments of the `main` functions.

Suggestion if you wish to dig further : In class we have seen that higher order methods have better rates of convergence, as the mesh size tends to zero and at comparable total number of degrees of freedom, provided the original solution itself (i.e. the one without FEM approximation) is sufficiently regular. You might wish to test that claim numerically on a Poisson problem. I'd suggest to work of a flat torus (i.e. a 2D planar grid with periodic boundary conditions) rather than the sphere. First because in the case of the sphere there is also the effect of the mesh itself approximating the smooth manifold. Second because you can then more easily devise right-hand-sides f for which you can compute the solution explicitly on paper. To exhibit the order of convergence, use a plot of L^2 and H^1 errors with respect to mesh size, using logarithmic scales on the axis.

Project 2 : Adding a boundary and boundary constraints

The goal of this project is to adapt our solvers for cases of 2D surface meshes with a boundary, an corresponding appropriate boundary conditions.

In the case of the Poisson solver, you might implement non homogeneous Dirichlet boundary conditions, since it is the most interesting case from the point of view of implementation. Identifying which vertices of the mesh are on the boundary is a prerequisite. You may assume that the mesh is oriented and manifold, so that a boundary edge is an edge with exactly one incident triangle, and a boundary vertex is simply a vertex belonging to a boundary edge.

The case of the Navier-Stokes solver in vorticity/stream function framework requires some thinking at the theoretical level first, regarding which boundary conditions are physically relevant. Recall that the velocity u of the fluid and the stream function Ψ were related by the equation

$$u = \nabla^\perp \Psi.$$

A natural set-up in the presence of boundaries is to assume a non-penetrating condition, that is

$$u \cdot n = 0 \text{ on } \partial\Omega,$$

where here n denotes the normal to the boundary. This can be rewritten as

$$\nabla^\perp \Psi \cdot n = 0, \quad \text{or equivalently} \quad \nabla \Psi \cdot \tau = 0,$$

where here τ is a tangent vector to the boundary. The latter implies that Ψ is constant on each connected component of $\partial\Omega$. In the project you may assume that $\partial\Omega$ is connected (as is the case e.g. for Ω a half-sphere surface mesh, hence only slightly modifying our class framework, or even more simply a 2D square grid). In that case, since Ψ is only also globally defined up to a fixed constant (adding a constant to Ψ does not modify the physical quantity u), you can assume that $\Psi \equiv 0$ on $\partial\Omega$. Therefore, for the Poisson solver part of the resolution, you will simply have

$$\Delta \Psi = \omega \text{ on } \Omega, \quad \Psi = 0 \text{ on } \partial\Omega.$$

Suggestion if you wish to dig further : Try discussing the boundary conditions for ω that we are implicitly assuming here by not modifying the weak formulation for ω with respect to the case without boundary. Other more physically relevant conditions might impose the value of the tangential part of the velocity (e.g. a no-slip condition corresponding to $u \cdot \tau = 0$, which in addition to the impermeability condition yields $u = 0$ on $\partial\Omega$), or a so-called Navier type condition. Both of these are mathematically more challenging (non local) in the context of the vorticity-stream formulation.

Project 3 : Adding a Coriolis force term

The goal of this project is to adapt our Navier-Stokes solver on the sphere in order to incorporate a Coriolis force term.

For the report it is expected that you recall what the Coriolis force is and how it translates into the formulation of the Navier-Stokes equations, and specifically in our case in the context of the vorticity / stream function formulation.

The literature you might find online will most probably rely on using spherical coordinates, despite their singularity at the poles. In class though, we have noticed that the vorticity / stream function formulation can be written intrinsically. The same is true for the Coriolis term, and if you are facing difficulty to obtain it I'd suggest looking at equation (11) of that nice 4 pages paper¹ from the late eighties by Ilyin and Filatov. Although it also uses spherical coordinates, you should recognize all of our terms (in particular our transport term T is called J there), and the slight modification due to the Coriolis force (called ℓ). Pay attention that what is denoted by ω in the paper is not what we called vorticity, it is the magnitude of the earth rotation speed. When testing, you may try to use numerical values corresponding to reality, or exaggerating it to strengthen its effect. Pay attention to the scales and units though (our “earth mesh” has been rescaled to the unit S^2 sphere). Any simulation of yours (that includes devising an initial vorticity) that vaguely resembles a typical atmospheric circulation would be highly appreciated !

¹It is written in russian but you only need to read the math, which as you know is a universal language :-)

Project 4: Elimination tree and direct sparse Cholesky solver

For this project you will write your own Cholesky factorization code

$$A = LL^T$$

for SPD sparse matrices A in the CSR format (or CSC if you prefer columns over rows).

You will then use it instead of the conjugate gradient for our Navier-Stokes solver. The factorization will be done before the simulation starts, and only forward and backward substitutions will be required at each time step.

As we have seen in class one key step of this factorization in the framework of sparse matrices is to determine in advance which matrix entries in L might end-up being non zero (the fill-in), because these may be much more than in A and it is necessary or at least useful to “prepare” L (reserve memory etc) before filling it. This phase is usually called the *symbolic* phase, because the potential non zero entries are located and counted in each row, but are not yet computed.

Since the theoretical background behind the process requires a bit of thinking, you are suggested to use as a reference the book *Algorithms for Sparse Linear Systems* by J. Scott & M. Tuma (in open access, already uploaded in the git repo inside the “notes” directory). Mostly relevant to us here are sections 4.2, 4.3, (5.1) and 5.2. In particular you will learn the notion of *elimination tree*, and you will directly translate algorithms 4.2 and 4.3 (for the symbolic phase) and then 5. x (choose $x \in 5, 6, 7$ most adapted to your set-up, for the actual factorization) into code. For the factorization step, it is important to avoid computing elements of L which were not marked as potential non zeros in the symbolic phase, but this actual factorization phase will remain the most computationally expensive one.

Suggestion : Already after the symbolic phase, you may test using your code (e.g. computing the amount of fill-in) the importance of the ordering of the unknowns, as we discussed in class (see also section 8.4 of the book). For that purpose you can freely use the `src/mesh/cube2.cpp` file that implements the cube mesh with nested dissection ordering “by hand”. For the application to Navier-Stokes, only keep the nested dissection version for better performance.

Project 5 : A (convex) nonlinear elliptic problem

Let Ω be a bounded domain in $\mathbb{R}^2$ and let $f : \Omega \rightarrow \mathbb{R}$. The area of the graph of f is given by

$$A(f) := \int_{\Omega} \sqrt{1 + |\nabla f|^2}.$$

The graph of f is said to be a minimal surface / minimal graph if

$$A(f) \leq A(g) \text{ for any } g : \Omega \rightarrow \mathbb{R} \text{ s.t. } f = g \text{ on } \partial\Omega,$$

in other words if its area is minimal among all graphs assuming the same boundary curve. A minimal graph satisfies the minimal surface equation

$$\operatorname{div} \left(\frac{\nabla f}{\sqrt{1 + |\nabla f|^2}} \right) = 0 \text{ on } \Omega,$$

which is nothing but the Euler-Lagrange equation associated to $A(\cdot)$.

Given a planar 2D triangular mesh Ω_h , and piecewise affine function f over that mesh (hence representable using a $\mathbb{P}_1$ -Lagrange finite element basis), present and code an algorithm that will compute/converge to the (unique) minimal graph f_* in that same finite element space and assuming the same boundary values as f .

Hint : Note that the set of admissible candidates is an affine subspace of the $\mathbb{P}_1$ -Lagrange space over Ω_h , and the functional $A(\cdot)$ is convex on that set. Any reasonable gradient descent method should therefore converge.

Suggestion if you wish to dig further : In nature minimal surfaces (e.g. soap films) need not be graphs. The mathematical set-up required to study the so-called Plateau's problem (finding a minimal surface given a boundary curve) is therefore somewhat more complex, but the resulting Euler-Lagrange equation is simpler since it amounts to the homogeneous linear Poisson equation $\Delta f = 0$. It comes with additional nonlinear constraints though, and besides f is no longer a scalar function (above the graph was $z = f(x, y)$, in the parametric formulation $f(x, y, z) = (f_1(x, y), f_2(x, y), f_3(x, y))$ is a vector valued function. Solutions (they need no longer be unique) may still be searched for using a P1 finite element methods, you might for example try to reproduce the strategy explained in that paper, in section 5.

Project 6 : Using existing software

Understanding the math and the difficulties/challenges associated to the implementation of finite element methods is important in order to later build on solid grounds. Since the subject is broad, it is also helpful if not important to be able to reuse (and potentially improve or make evolve) existing code bases. For this last subject proposal, the goal is to reproduce the 2D Navier-Stokes solver that we coded from scratch, by using third party libraries/software for each step (meshing, build-up of matrices and solution of linear systems). You might for example choose one among the following well established open source projects :

- **FreeFem** <https://freefem.org>
More a software than a library, it comes with its own scripting language leading to short single file scripts. In particular, you won't write any C/C++ code if choosing it.
- **FEniCS** <https://fenicsproject.org>
Or it's DOLPHINx Python wrapper, if you also wish to avoid C/C++.
- **Deal.II** <https://www.dealii.org>

Demos and tutorials exist for all three, they might be easier to begin with w.r.t. reading their documentation. The first one, FreeFem, is developed here at SU.

Another project direction, equally interesting, would be to only replace our linear system solver(s) by state of the art ones. I'd strongly suggest getting access to the **MUMPS** solver <https://mumps-solver.org> (award winning and also made in France)